



embryonaut

HECKS · BROWNFIELD

Bubble context

A clean model for one slice of the business, wrapped around the systems a company already runs, and grown until the old ones can go.



WHAT THIS COVERS

Using Hecks with legacy systems

The idea

What a bubble is, and why it is a chapter and not a new construct.

The parts

Bluebook, hecksagon and world, and who supplies each.

The path

Five steps from watching legacy to retiring it.

The plumbing

Prewritten adapters that are thin wrappers over shared clients.

The help

Importer, shadow mode and a drift report, beyond adapters.

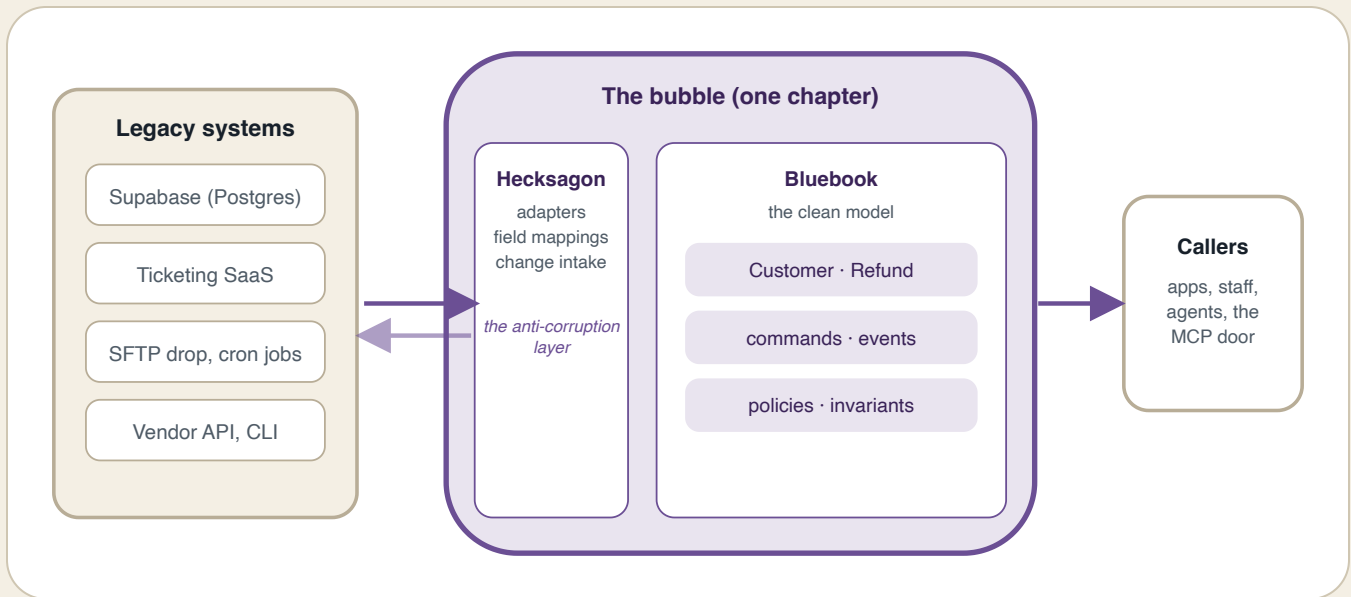
The decisions

What is settled and what still needs an answer.

THE IDEA

A bubble is a chapter wrapped around legacy

Evans called it a bubble context: a small, clean bounded context built inside a legacy system, protected by an anti-corruption layer. In Hecks it needs no new keyword. It is a chapter, loaded into a host like any other, whose hexsagon binds to the legacy systems.



- **Legacy keeps working.** Nothing in it has to change for the first step. The bubble sits beside it.
- **The model stays clean.** Legacy field names, table shapes and vendor quirks stop at the hexsagon and never reach the bluebook.
- **Callers move one at a time.** Each legacy entry point is repointed at the bubble's commands when it is ready, which is the strangler pattern.
- **Several bubbles, one host.** Refunds, Onboarding and Inventory are separate chapters that talk through **translates**, **uses_framework** and cross-domain policies.

THE PARTS

Given a bluebook, supply adapters and settings

Building an integration should mean writing no application code. A company takes a bluebook (written once, shared) and supplies two things of its own.

PART	HOLDS	WHO SUPPLIES IT
Bluebook	The clean model and its ports: the aggregates, commands, events and the outside services it needs, named as ports. Says nothing about any one company.	Shared, vendored, versioned
Hecksagon	Which adapter fulfils each port, and the field mappings from legacy shapes to the bluebook's language. Inbound adapters that watch legacy live here too.	The company
World	Per-environment settings: base URLs, tenant ids, credential references (never values), rate limits, and the read-only switch. Dev, staging and prod are different worlds over one hecksagon.	The company, per environment

Check before boot. The model check refuses a hecksagon that leaves a port unbound or binds an adapter to a port it does not implement, and refuses a world that lacks a setting an adapter declares as required.

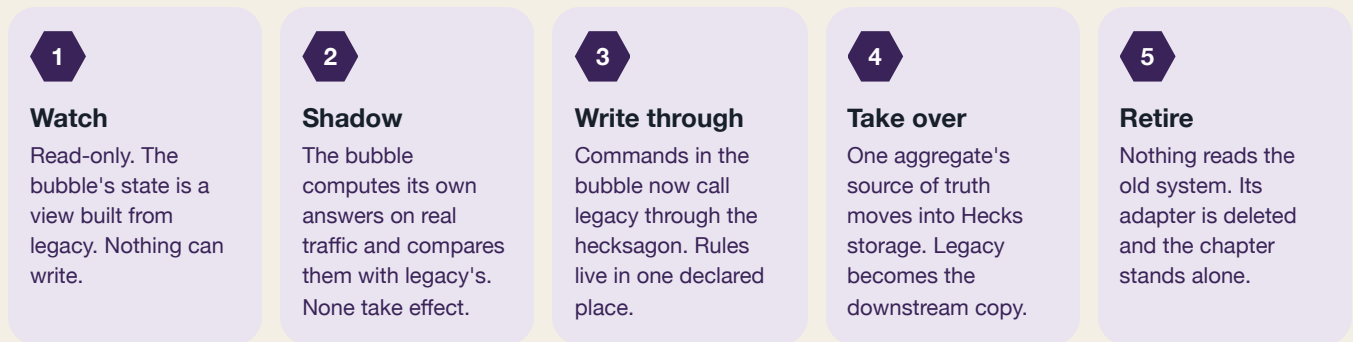
The words, and the industry terms they stand for

IN HECKS	INDUSTRY TERM	WHERE IT COMES FROM
A bubble (a chapter with legacy-facing hecksagon)	Bubble context	Evans, Domain-Driven Design
The hecksagon's legacy-facing adapters and mappings	Anti-corruption layer	Evans, Domain-Driven Design
Inbound adapters that only watch	Wire tap, change data capture	Enterprise Integration Patterns
Moving callers across a step at a time	Strangler fig	Fowler
Port, adapter	Ports and adapters (hexagonal)	Cockburn

Only **bubble** is a new word, and it is used in prose. In code and files there is nothing new: it is a bluebook, a hecksagon and a world.

Five steps from watching to retiring

A company can stop at any step and still have something useful. Each step adds trust before it adds risk.



What the company gets along the way

- One **vocabulary** over systems that each name things differently.
- One **audit trail** across systems that had none.
- Swappable vendors.** Moving from one payments or CRM provider to another is an adapter change.
- Safe agent access.** An agent works through declared commands and never touches the legacy database.
- Rules in one place**, in place of stored procedures, spreadsheets and cron scripts.

Where it gets hard

- Dirty data.** Rows that fit no aggregate need a reviewable quarantine with a reason, not an error.
- Two writers.** While both systems accept changes, a cutover needs a written procedure. The transactional outbox gives retries an owner.
- Systems that change on their own.** A legacy app that writes straight to its database needs polling, change capture or webhooks, or the bubble drifts.
- Modeling discipline.** A bluebook that mirrors the legacy schema gains nothing.

Read-only is a setting, not a promise. Step 1 depends on the adapter client refusing writes when the world says read-only, and on the model check confirming that a read-only adapter exposes no write operations.

Prewritten adapters, as thin wrappers

Each adapter calls a shared client and adds only what is particular to its service. The per-service part is data, so it is written once and read by both the Ruby and Rust runtimes.

Shared transport clients

- Http.** Base URL, auth, retries, rate limits, pagination, error mapping.
- Cli.** Argv, environment, exit codes, output parsing. Builds on the existing Shell adapter.
- Protocol.** SQL, SFTP and files, SMTP, SOAP, queues and webhooks. These are the workhorses of brownfield, since legacy mostly speaks them.

Per service

- A declarative mapping from a port ask to a request: method, path, body, response fields, auth scheme.
- Credentials by reference, resolved at run time and never on disk.
- A recorded-response fixture checked against the conformance corpus in both runtimes, so drift fails the build.
- An escape hatch for services that do not fit the table, flagged as a known parity gap until ported.

PORT	SERVICES
Payments	Stripe, PayPal, Square, Adyen, Paddle
Email, SMS	Resend, SendGrid, Postmark, Mailgun, SES, SMTP/IMAP; Twilio, Vonage, MessageBird
Chat, alerts	Slack, Discord, Teams, Telegram, PagerDuty
Source, CI	GitHub, GitLab, Bitbucket, CircleCI, gh and glab
Tickets, work	Linear, Jira, Asana, Trello, Notion, Shortcut, ServiceNow
CRM, support	Salesforce, HubSpot, Zendesk, Intercom, Pipedrive, Dynamics
Accounting, ERP, HR	QuickBooks, Xero, NetSuite, SAP, Oracle, Workday; Shopify, WooCommerce
Files, workspace	S3, GCS, Azure Blob, Dropbox, Drive, Box, SFTP, CSV; Gmail, Calendar, Sheets, Microsoft Graph
Identity	Auth0, Okta, Clerk, OAuth and OIDC, LDAP and Active Directory
Cloud, CLI	aws, gcloud, az, kubectl, terraform, docker, vercel, fly, Cloudflare
Data	Postgres, MySQL, MongoDB, Redis, Airtable, Supabase, Firebase, generic SQL; Snowflake, BigQuery
Events	Kafka, RabbitMQ, SQS and SNS, inbound and outbound webhooks
Observe, search, AI, maps	Datadog, Sentry, Grafana, Umami, PostHog; Algolia, Elastic; Anthropic, OpenAI; Google Maps, Mapbox
Legacy protocols	SOAP and WSDL, EDI

WORKED EXAMPLE

A Supabase adapter DESIGN SKETCH, NOT BUILT

Supabase is the first service we would wrap, because one project is a legacy database, a change feed, an identity provider and a file store behind a single key. It exercises every part of the design.

SUPABASE GIVES	REACHED THROUGH	WHICH BUBBLE PORT
Postgres tables	REST (/rest/v1) or SQL	Persistence and read models
Realtime, database webhooks	Change feed, HTTP callbacks	Inbound change events (the role a tap plays)
Auth	/auth/v1	Identity
Storage	/storage/v1	Files
Edge functions	/functions/v1	Outbound asks

1. The adapter declaration

The same two-file shape every adapter has today: this file says what it binds and which settings it needs.

```
Hecks.adapter "Supabase" do
  port "persistence"
  field :url
  secret :service_key
  field :read_only
end
```

2. The hexsagon binds the model

The clean aggregate is bound to the company's existing tables.

```
Hecks.hexsagon "Refunds" do
  Refunds::Refund.persisted_by("Supabase")
end
```

3. The world supplies the environment

URL and mode vary by environment. The key is a secret, read from the environment and never written in a file.

```
url:          the project's API URL
service_key:  from the secret store
read_only:    true      # step 1
```

What the shared client does

- ✦ Sends the **apikey** and bearer headers, filters with query operators, and pages with ranges.
- ✦ Refuses every write while **read_only** is true.
- ✦ Maps table columns to aggregate fields from the hexsagon's mapping, so legacy names stop here.

Open point this example raises. The persistence port is append-before-project, and an existing table has no journal. For step 1 the bubble should read the table as a projection, and a table that takes writes needs a different port or a journal built beside it. Settle this before the first slice.

Beyond adapters, and what is open

More ways to help a company move

- **Importer.** Point it at a schema, an OpenAPI spec, a CLI's help or a CSV export and it drafts a chapter and hexsagon for a person to edit. This removes the blank page.
- **Shadow mode.** Real traffic, compared answers, no effect. A mismatch report is a plain go or no-go signal.
- **Drift report.** What the bubble holds against what legacy says, on demand.
- **Characterization capture.** Record real inputs and outputs from legacy and turn them into conformance fixtures.
- **Rules extraction.** Surface candidate invariants from stored procedures and scripts for a person to approve.
- **Reference bubbles.** Prebuilt chapters for common domains, so a company writes only its hexsagon and world.
- **Cutover playbook.** Generated per bubble, with the checks and rollback for each step.

Build order

- 1. Port declarations, adapter-to-port checks and typed world settings in the model check.
- 2. Transport clients and the data-driven adapter format, Ruby and Rust.
- 3. First slice: Stripe, Slack, gh, generic SQL and webhooks, with fixtures.
- 4. The rest of the catalog.
- 5. Importer, then shadow mode and the drift report.

Settled

- A bubble is a chapter plus its hexsagon, not a new construct.
- The model check guards the generated wiring.
- Adapters are thin wrappers over shared clients.

Open. Whether the field mappings live in the hexsagon (the recommendation, so the hexsagon is literally the translation layer) or in the world. Whether each chapter carries its own hexsagon (recommended) or the host has one. How read-only is enforced and checked. Whether the catalog and transport layer keep these working names.